KENNESAW STATE
UNIVERSITY

CS 4277
Autoencoder

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

College of Computing and Software Engineering

Learning Objectives

- Principal components analysis
- Applications using AEs
- Deep AEs
  - Denoising AE
  - Sparse AE
  - Stacked AE
- Modern applications using AEs

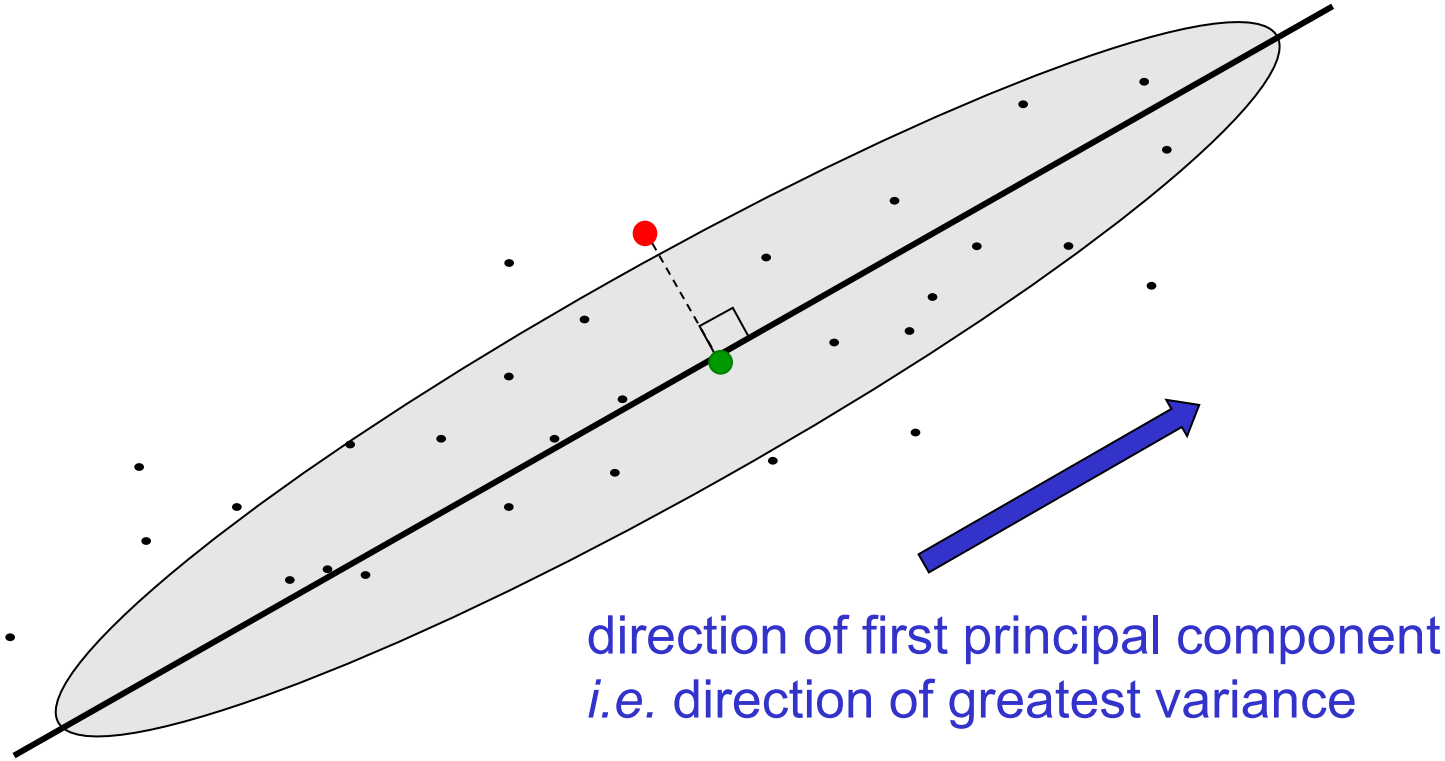
From Principal Components Analysis to Autoencoders

Principal Components Analysis

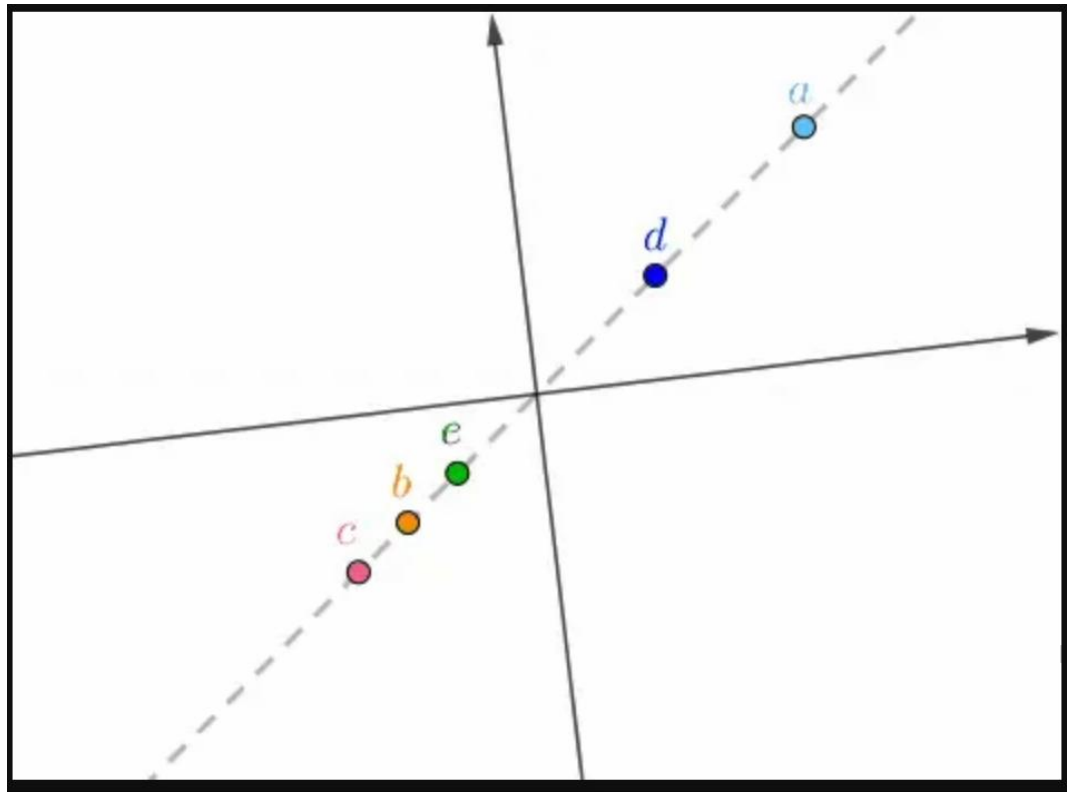
- This takes N-dimensional data and finds the M orthogonal directions in which the data have the most variance.
  - These M principal directions form a lower-dimensional subspace.
  - We can represent an N-dimensional datapoint by its projections onto the M principal directions.
  - This loses all information about where the datapoint is located in the remaining orthogonal directions.
- We reconstruct by using the mean value (over all the data) on the N-M directions that are not represented.
  - The reconstruction error is the sum over all these unrepresented directions of the squared differences of the datapoint from the mean.

A picture of PCA with N=2 and M=1

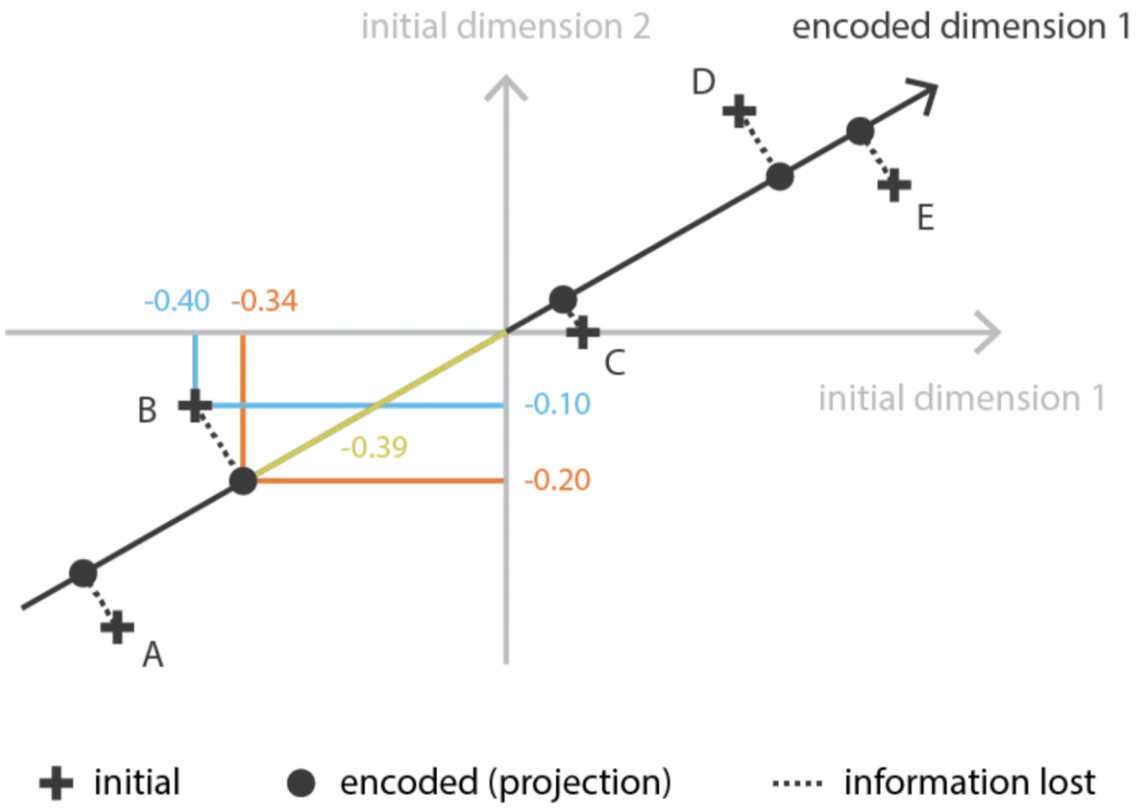
- The red point is represented by the green point. The reconstruction of the red point has an error equal to the squared distance between red and green points.



PCA example



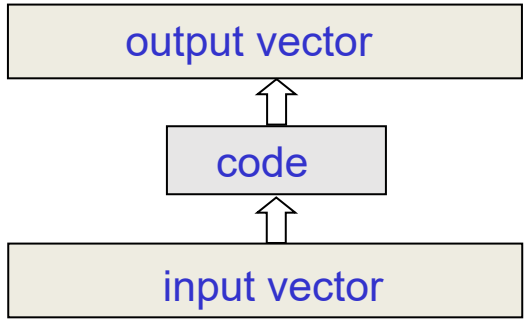
PCA is looking for the best linear subspace using linear algebra



Point	Initial	Encoded	Decoded
A	(-0.50, -0.40)	-0.63	(-0.54, -0.33)
B	(-0.40, -0.10)	-0.39	(-0.34, -0.20)
C	(0.10, 0.00)	0.09	(0.07 0.04)
D	(0.30, 0.30)	0.41	(0.35, 0.21)
E	(0.50, 0.20)	0.53	(0.46, 0.27)

Using backpropagation to implement PCA inefficiently

- Try to make the output be the same as the input in a network with a central bottleneck.

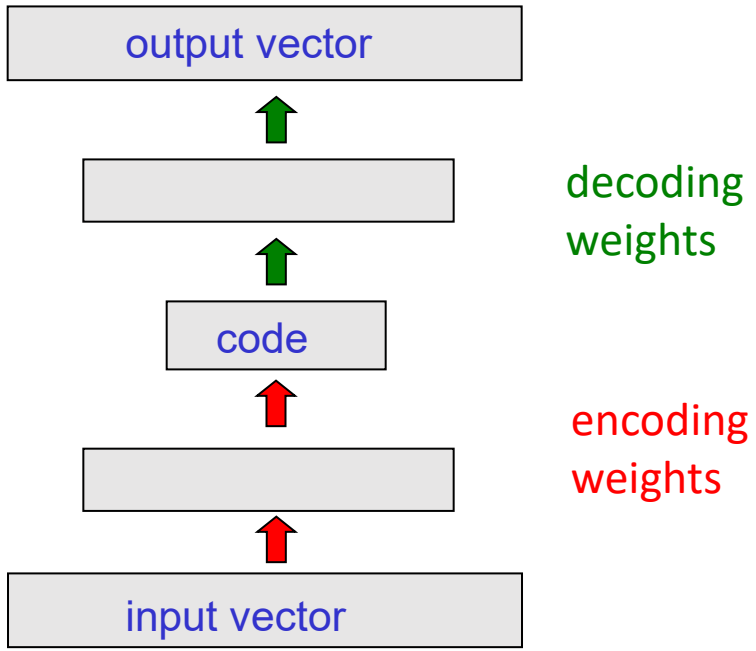


- The activities of the hidden units in the bottleneck form an efficient code.

- If the hidden and output layers are linear, it will learn hidden units that are a linear function of the data and minimize the squared reconstruction error.
  - This is exactly what PCA does.
- The M hidden units will span the same space as the first M components found by PCA
  - Their weight vectors may not be orthogonal.
  - They will tend to have equal variances.

Using backpropagation to generalize PCA

- With non-linear layers before and after the code, it should be possible to efficiently represent data that lies on or near a non-linear manifold.
  - The encoder converts coordinates in the input space to coordinates on the manifold.
  - The decoder does the inverse mapping.

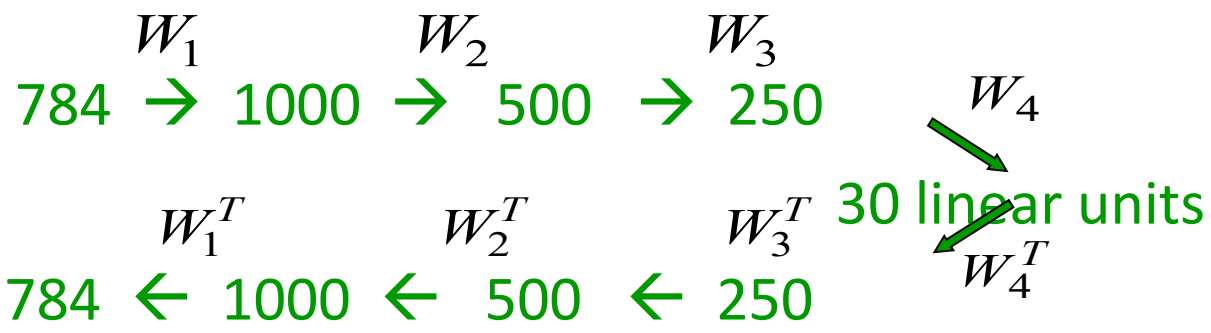


Application of Deep Autoencoders

Deep Autoencoders

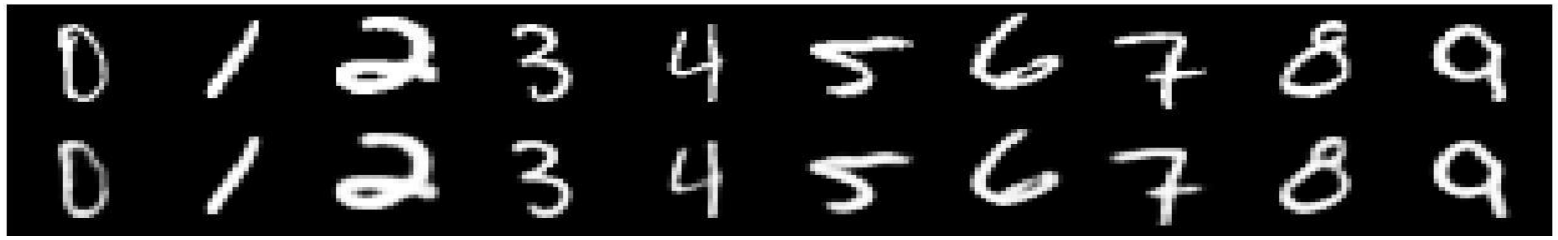
- They always looked like a really nice way to do non-linear dimensionality reduction:
  - They provide flexible mappings both ways.
  - The learning time is linear (or better) in the number of training cases.
  - The final encoding model is fairly compact and fast.
- But it turned out to be very difficult to optimize deep autoencoders using backpropagation.
  - With small initial weights the backpropagated gradient dies.
- We now have a much better ways to optimize them.
  - Use unsupervised layer-by-layer pre-training.
  - Or just initialize the weights carefully.

The first really successful deep autoencoders



1. train a stack of 4 Restricted Boltzmann Machine (RBM)'s and then "unroll" them.
2. Then fine-tune with backprop.

A comparison of methods for compressing digit images to 30 real numbers



real data

30-D deep AE

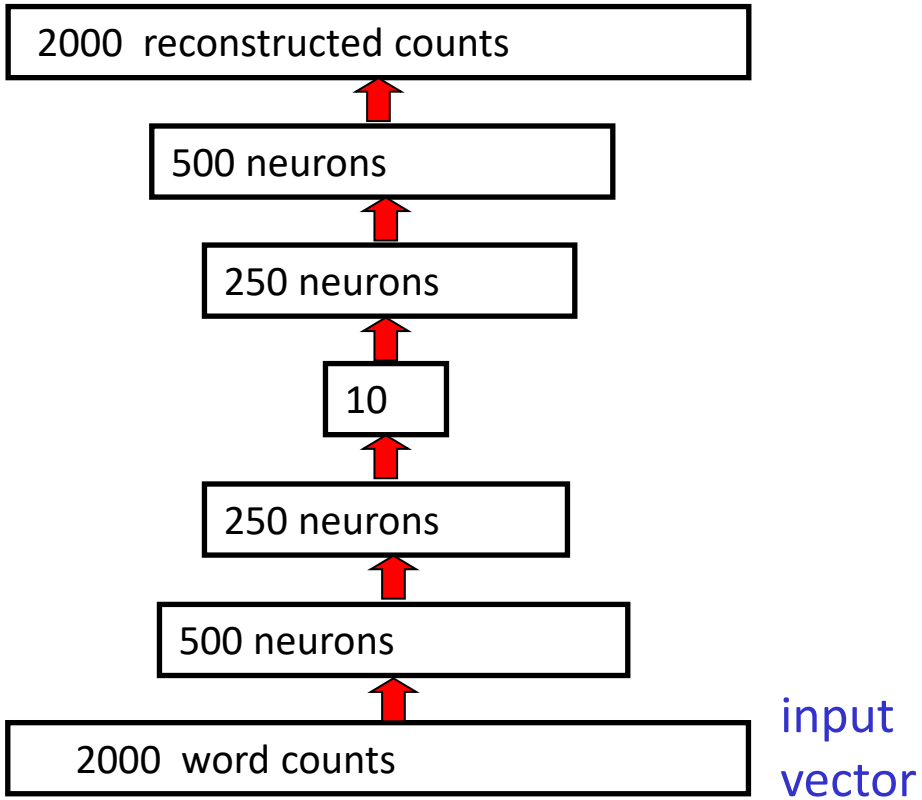
30-D PCA

Deep autoencoders for document retrieval and visualization

- How to find documents that are similar to a query document?
- Convert each document into a “bag of words”.
  - This is a vector of word counts ignoring order.
  - Ignore stop words (like “the” or “over”)
- We could compare the word counts of the query document and millions of other documents but this is too slow.
  - So we reduce each query vector to a much smaller vector that still contains most of the information about the content of the document.

0	fish
0	cheese
2	vector
2	count
0	school
2	query
1	reduce
1	bag pulpit
0	iraq word
0	
2	

How to compress the count vector



- Train the neural network to reproduce its input vector as its output
- This forces it to compress as much information as possible into the 10 numbers in the central bottleneck.
- These 10 numbers are then a good way to compare documents.

The non-linearity used for reconstructing bags of words

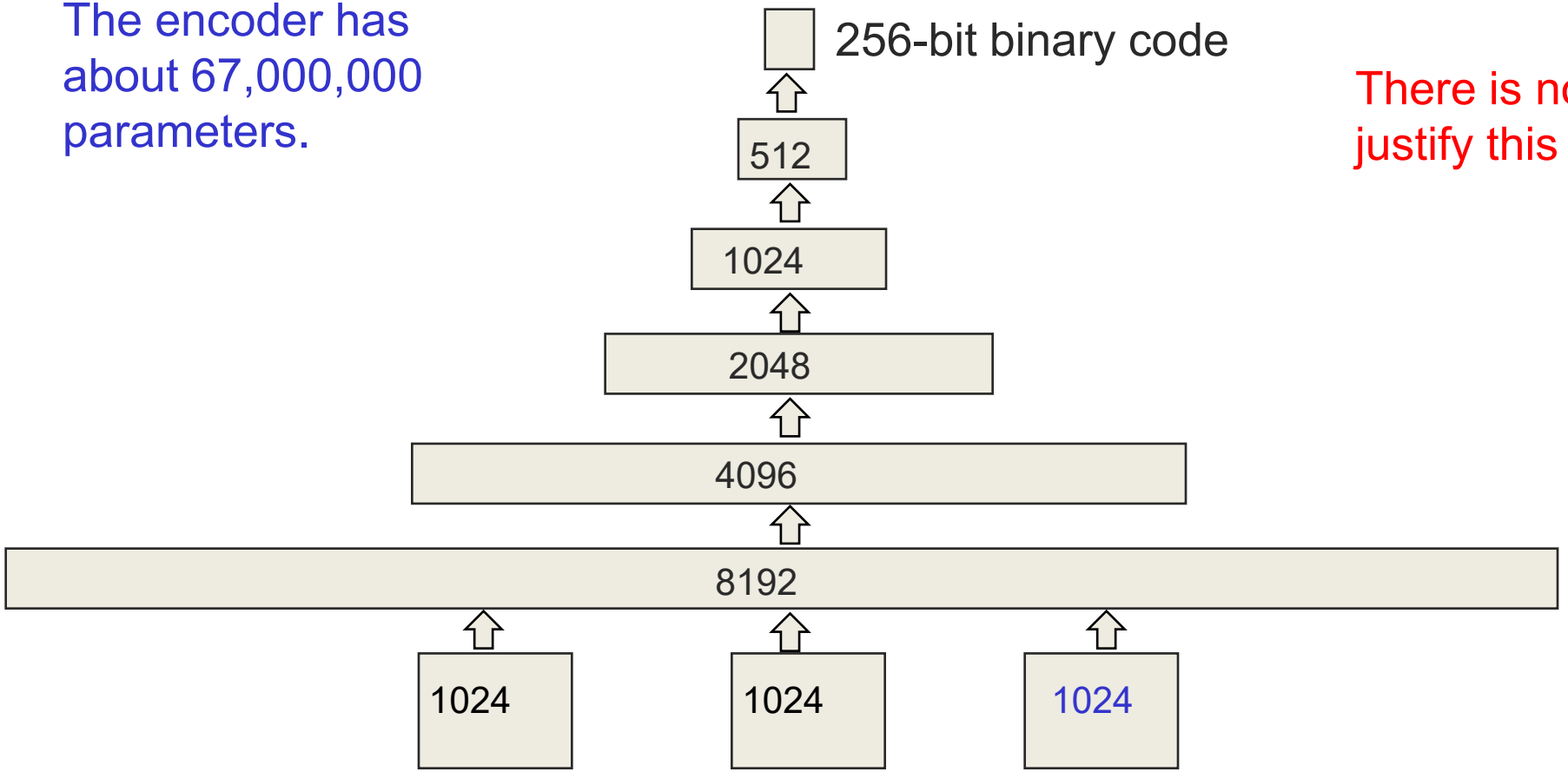
- Divide the counts in a bag of words vector by N, where N is the total number of non-stop words in the document.
  - The resulting probability vector gives the probability of getting a particular word if we pick a non-stop word at random from the document.
- At the output of the autoencoder, use softmax.
  - The probability vector defines the desired outputs of the softmax.

Learning binary codes for image retrieval

- Image retrieval is typically done by using the captions. Why not use the images too?
  - Pixels are not like words: individual pixels do not tell us much about the content.
  - Extracting object classes from images is hard (this is out of date!)
- Maybe we should extract a real-valued vector that has information about the content?
  - Matching real-valued vectors in a big database is slow and requires a lot of storage.
- Short binary codes are very easy to store and match.

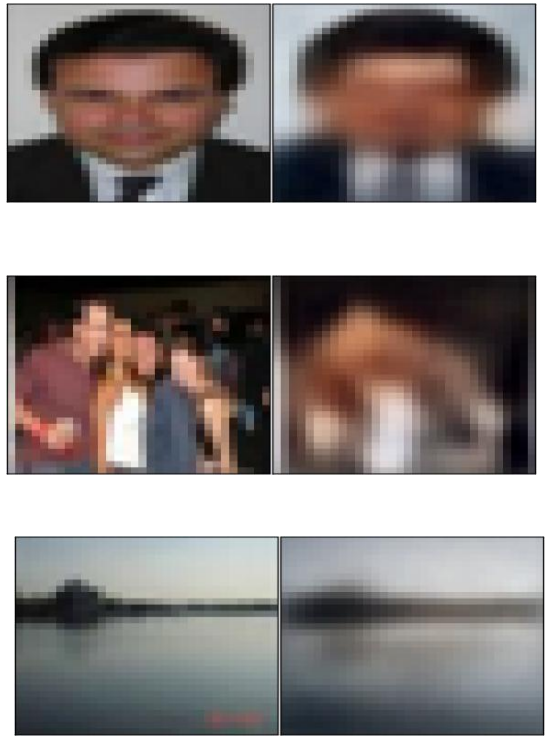
Krizhevsky's deep autoencoder

The encoder has about 67,000,000 parameters.



There is no theory to justify this architecture

Reconstructions of 32x32 color images from 256-bit codes



Images retrieved with AE

retrieved using 256 bit codes

retrieved using Euclidean distance in pixel intensity space

Images retrieved with AE

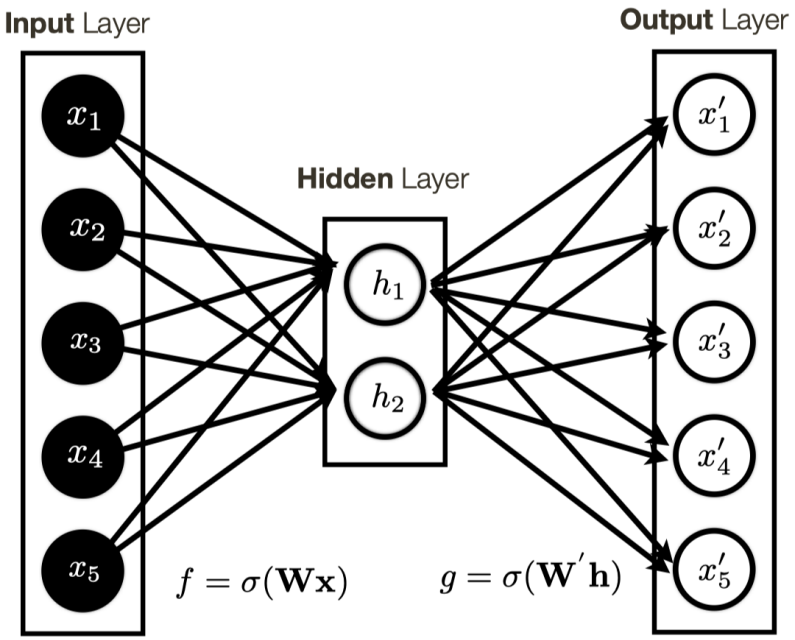
retrieved using 256 bit codes

retrieved using Euclidean distance in pixel intensity space

AEs

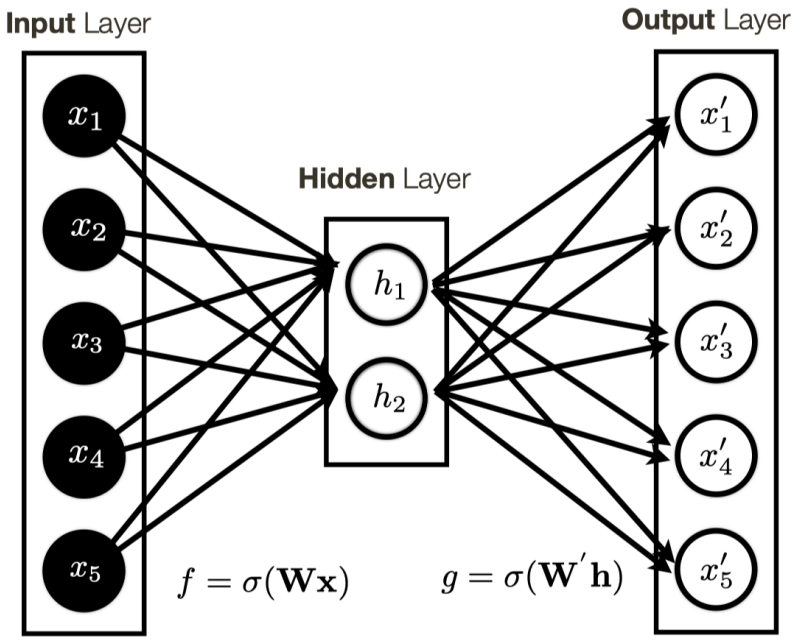
Autoencoder (AE)

- Feed forward network intended to reproduce the input
- Encoder/Decoder architecture
- Score function $\mathrm{x}' = f(g(x))$,



Autoencoder (AE)

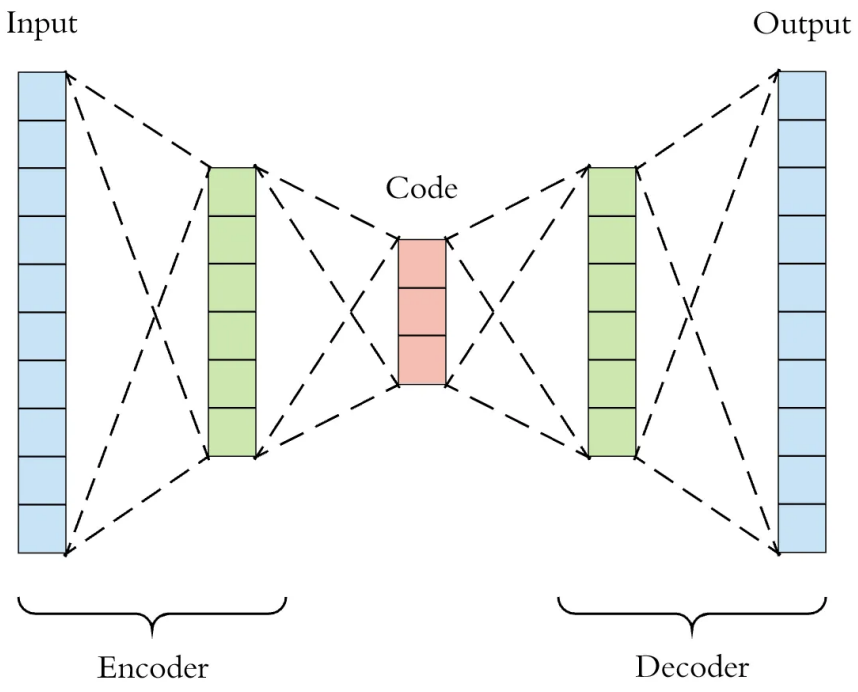
- A standard neural network architecture (linear layer followed by non-linearity)
  - $f(\cdot)$: encoder, e.g. multi-layered perceptron
  - $g(\cdot)$: decoder, e.g. multi-layered perceptron
- Activation depends on types of data (e.g., sigmoid for binary; linear for real valued)

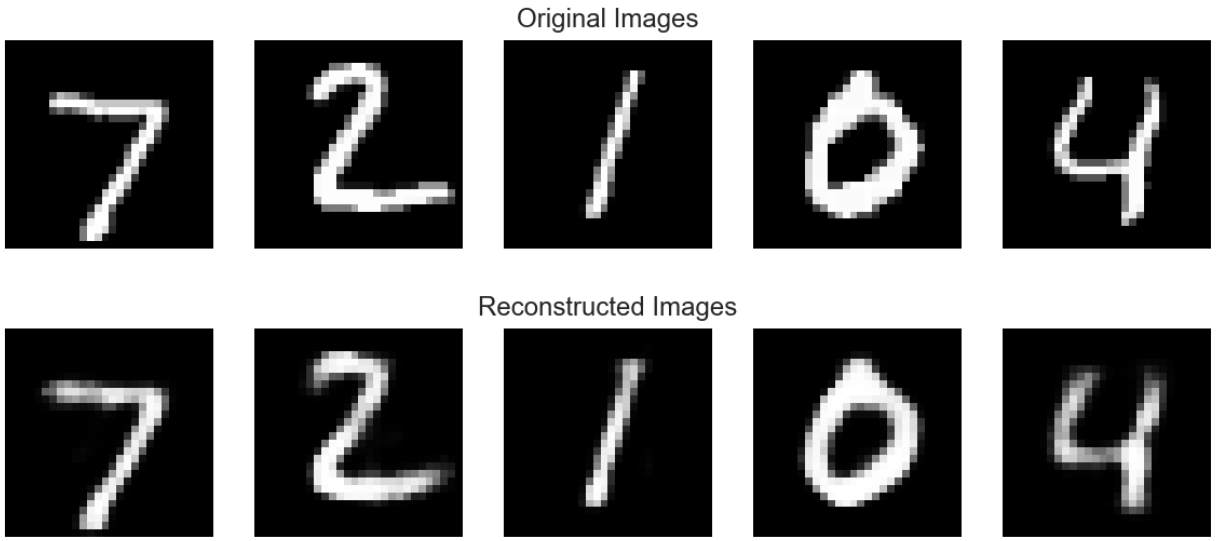


Autoencoders: Hidden Layer Dimensionality

- Larger than the input
  - No compression needed
  - Can trivially learn to just copy, no structure is learned
  - Does not encourage learning of meaningful features
- Smaller than the input
  - Will compress the data. Reconstruction of the data far from the training distribution will be difficult
  - Linear-linear encoder-decoder with Euclidian loss

Implementation on MNIST using AE

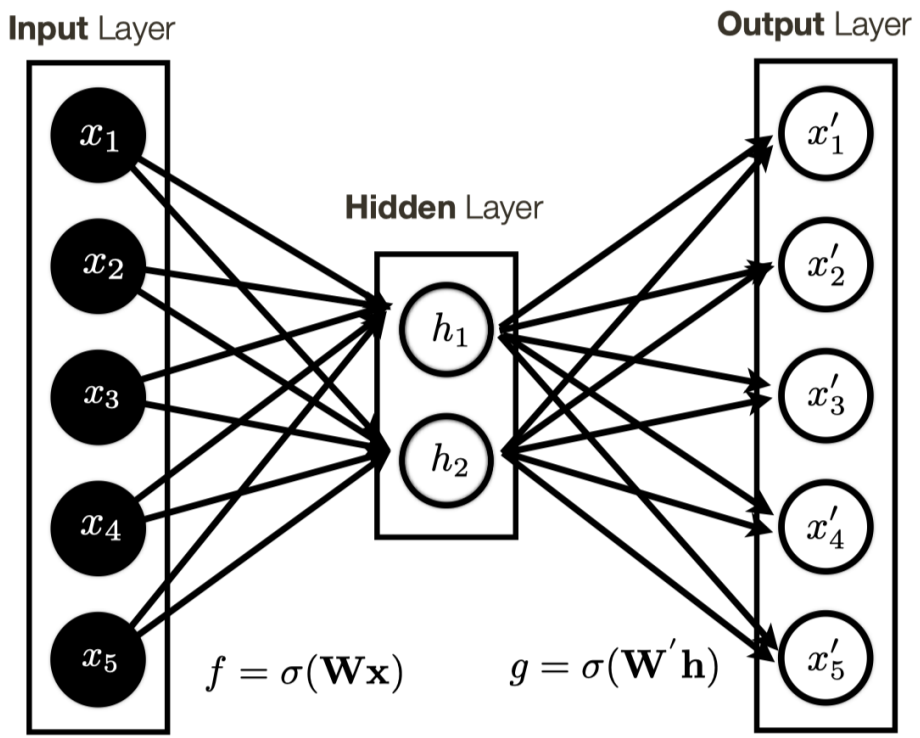




- The image size is 28x28 and we use them as a vector of 784 numbers between [0, 1]
- binary_crossentropy as loss function

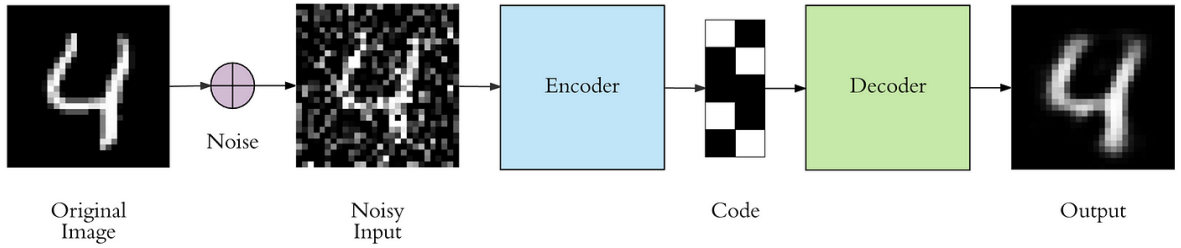
De-noising Autoencoder (DAE)

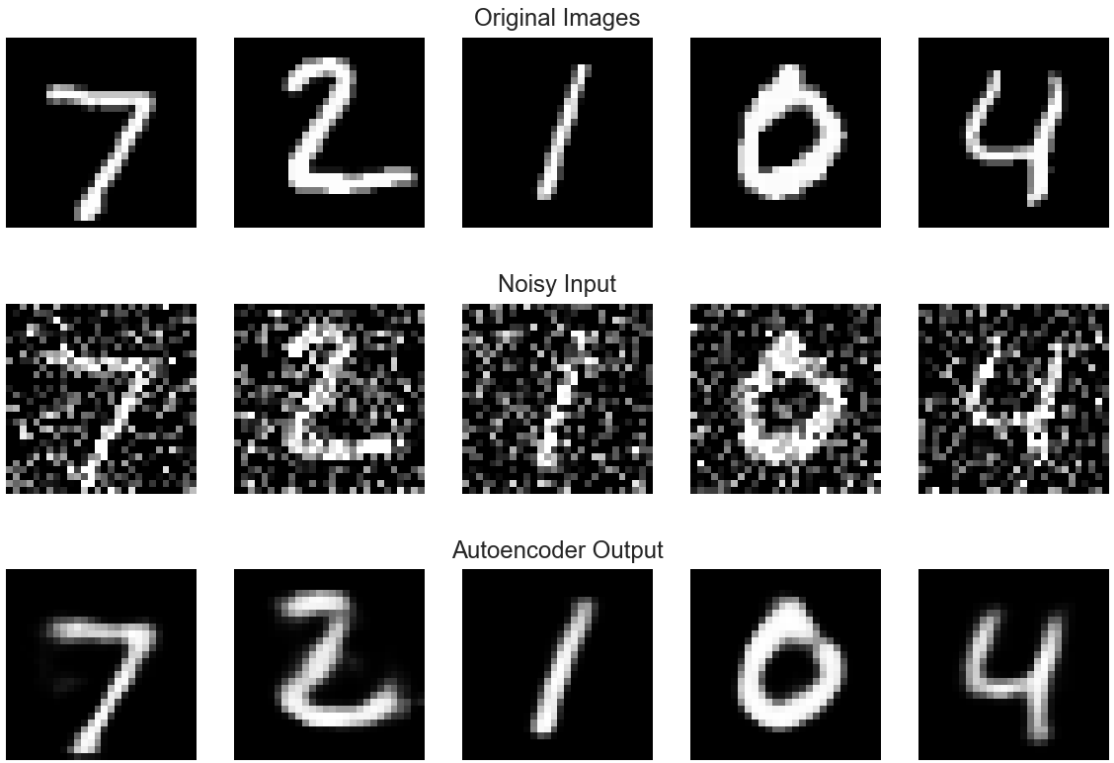
- Idea: add noise to input but learn to reconstruct the original
  - Leads to better representations
  - Prevents copying
- Note: different noise is added during each epoch



Implementation on MNIST using DAE

- Add random Gaussian noise to generate noisy input
- Workflow is shown below





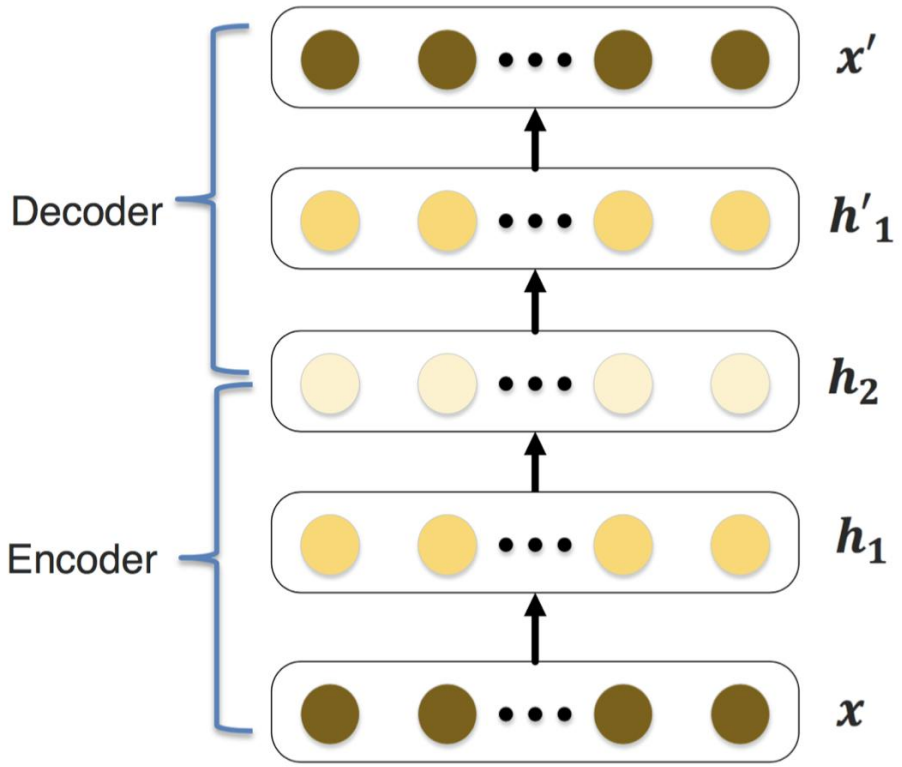
Sparse autoencoder

- Sparse autoencoder regularizes the autoencoder by using a sparsity constraint such that only a fraction of the nodes would have nonzero values, called active nodes
- Implementation: add a penalty term (a L2 regularization term) to the loss function such that only a fraction of the nodes become active

Original loss: $\mathrm{LOSS}_{\mathrm{mse}}$
New Loss: $\mathrm{LOSS}_{\mathrm{mse}} + \|W\|_2^2$

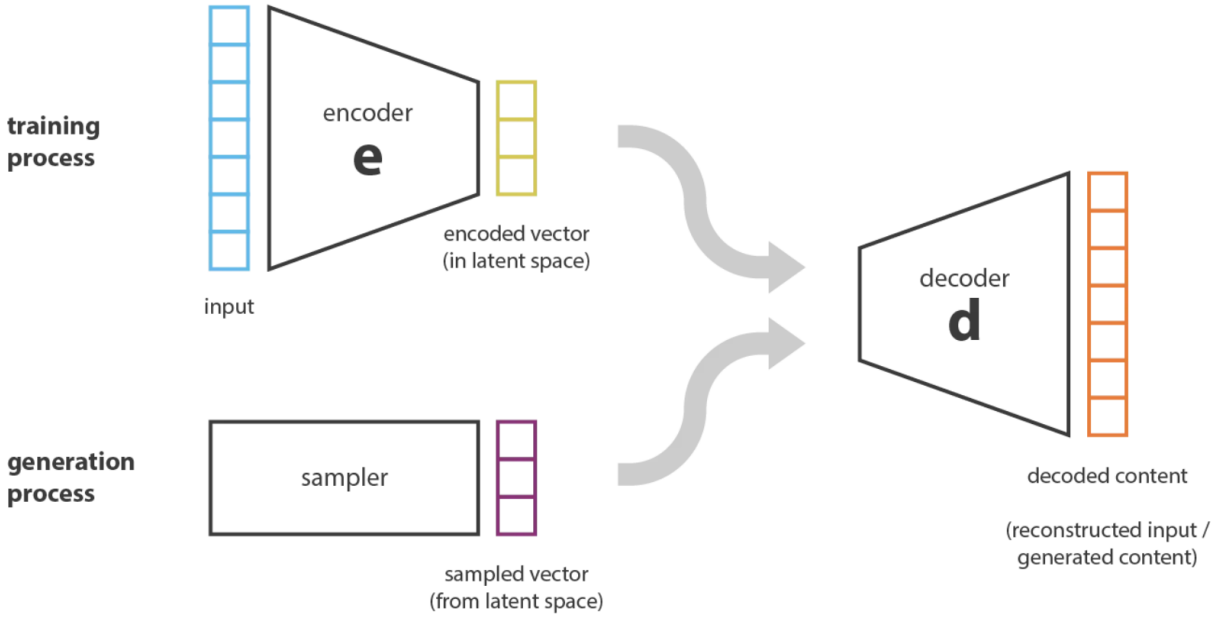
Stacked (deep) Autoencoders and Denoising Autoencoders

- What can we do with them?
  - Good for compression (better than PCA)
  - Disregard the decoder and use the middle layer as a representation
  - Insert layers between the input and middle layer or between the middle layer and output
  - Fine-tune the autoencoder for a task



Limitations of AEs for content generation

- A trained AE cannot produce any new content.



- Solution: generate new data by decoding points that are randomly sampled from the latent space.

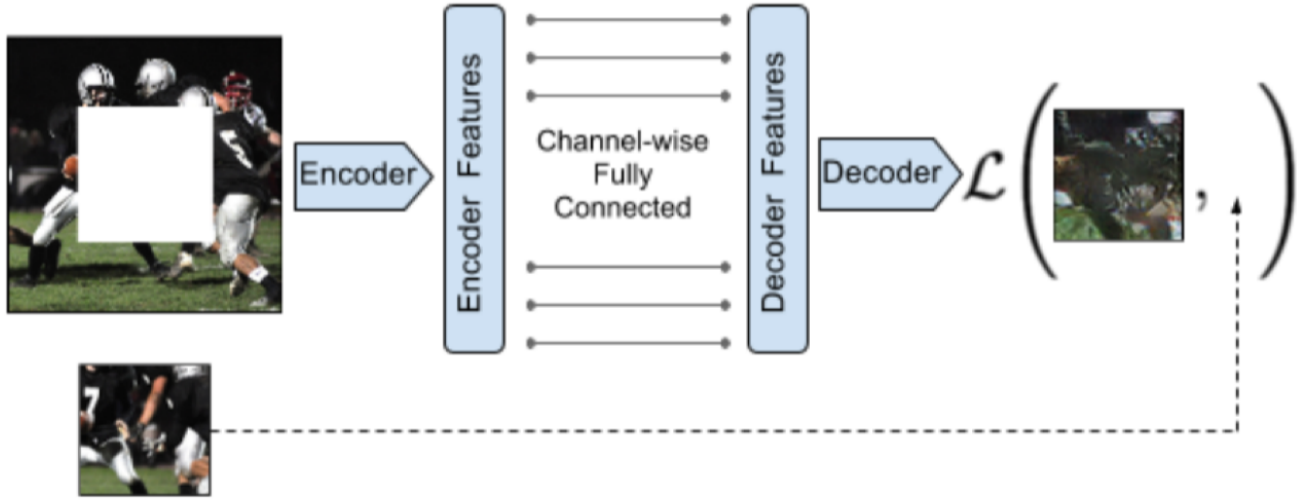
Hyperparameters

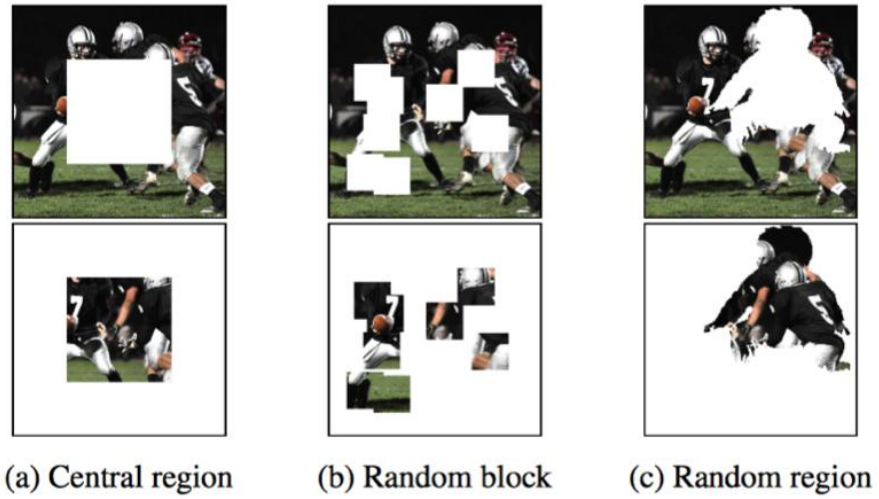
- Code size: number of nodes in the middle layer. Smaller size results in more compression
- Number of layers: the autoencoder can be as deep as we like (Stacked autoencoders)
- Number of nodes per layer
- Loss function
  - Mean squared error (MSE)
  - Binary cross entropy. (if the input values are in the range [0, 1])

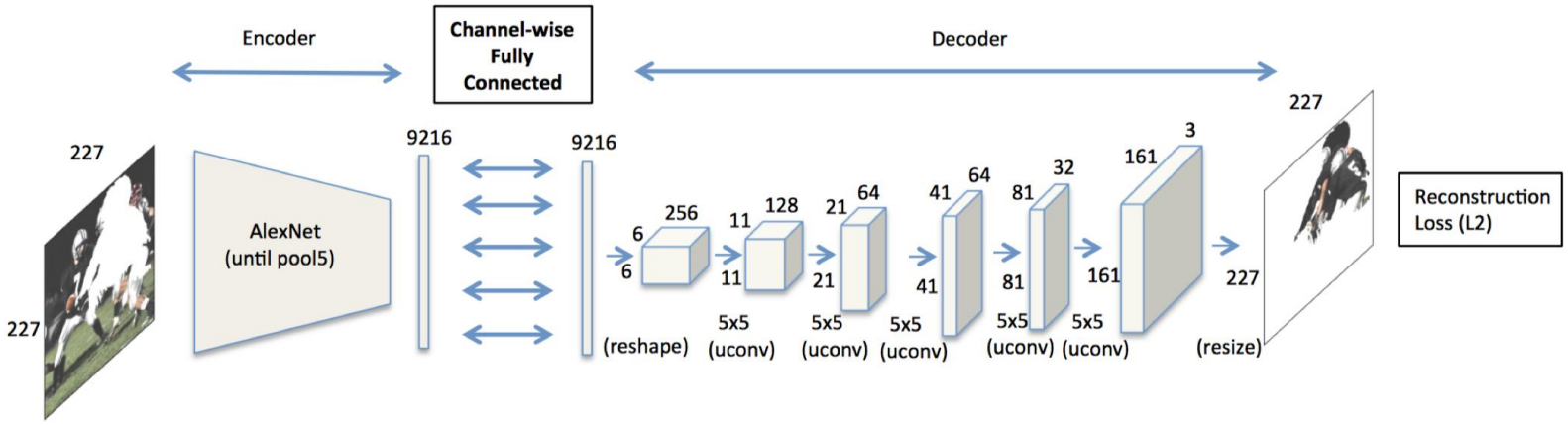
Modern applications using AE

Context Encoders

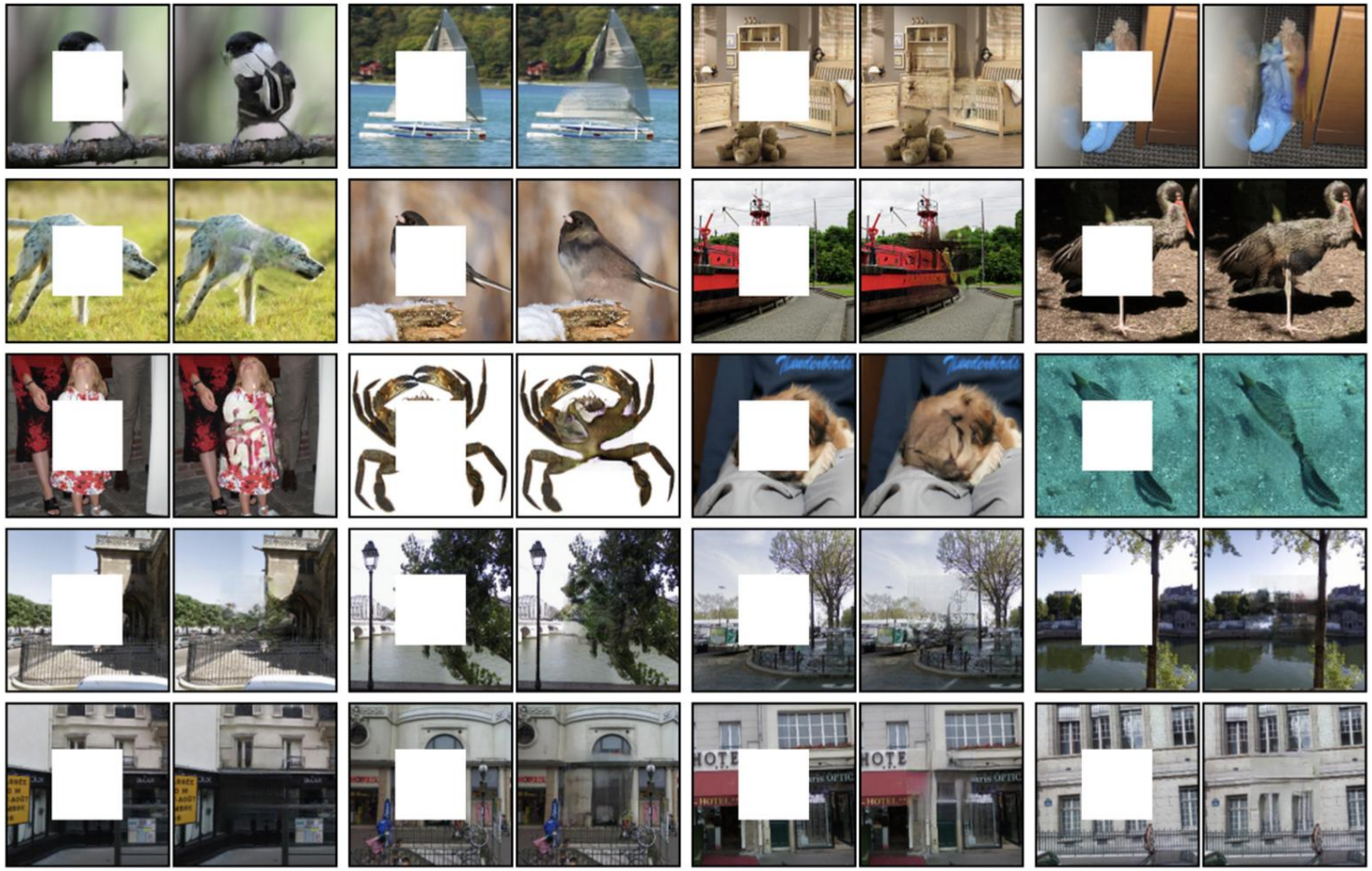
Pathak, D., Krahenbuhl, P., Donahue, J., Darrell, T. and Efros, A.A., 2016. Context encoders: Feature learning by inpainting. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2536-2544).





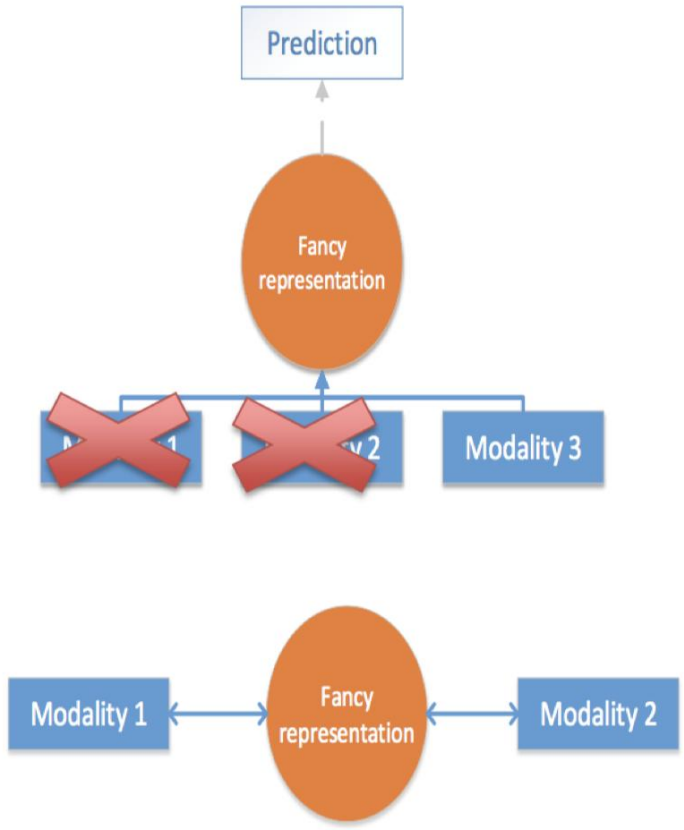


Context Encoders



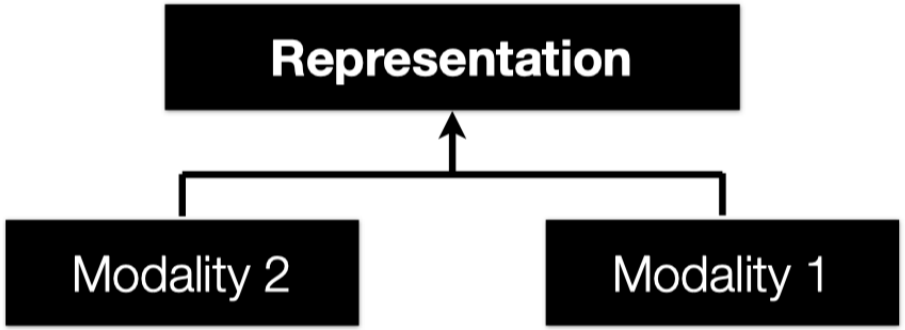
Multimodal Representations

- What is a good multimodal representation?
  - Similarity in the representation (somehow) implies similarity in corresponding concepts
  - Useful for various discriminative tasks (retrieval, mapping, fusion, etc.)
  - Possible to obtain in absence of one or mere modalities
  - Fill in missing modalities given others (map or translate between modalities)

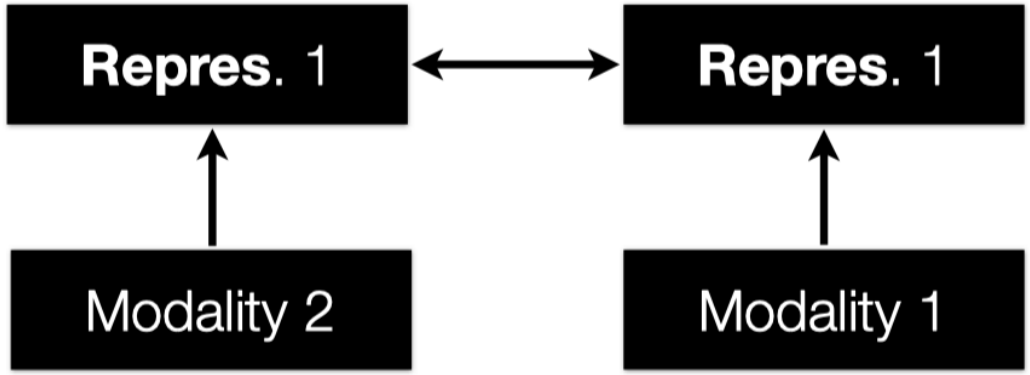


Multimodal Representation Types

- Joint representation
  - Simplest version: modality concatenation (early fusion)
  - Can be learned supervised or unsupervised

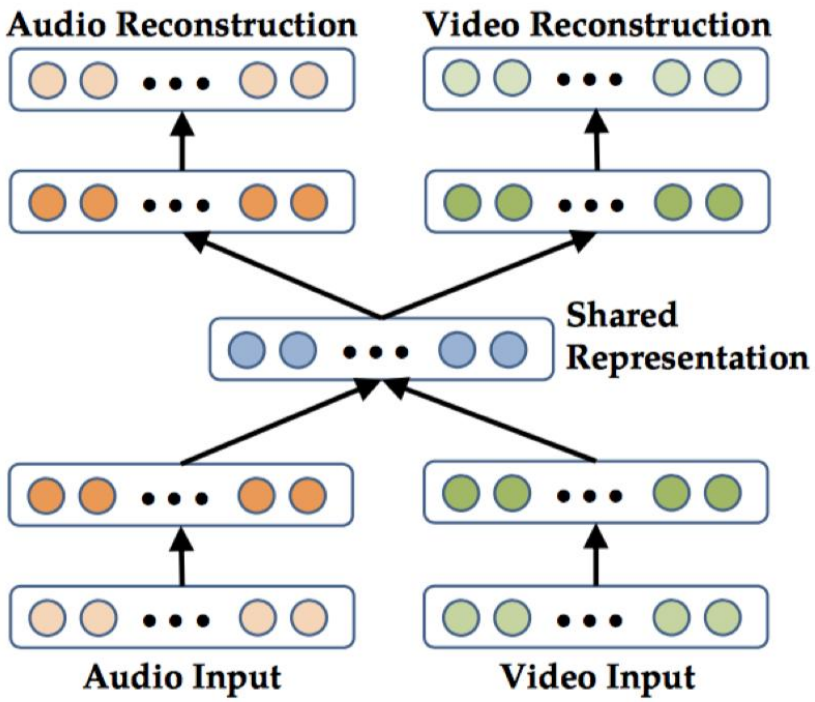


- Coordinated representations:
  - Similarity-based methods (e.g., cosine distance)
  - Structure constraints (e.g., orthogonality, sparseness)



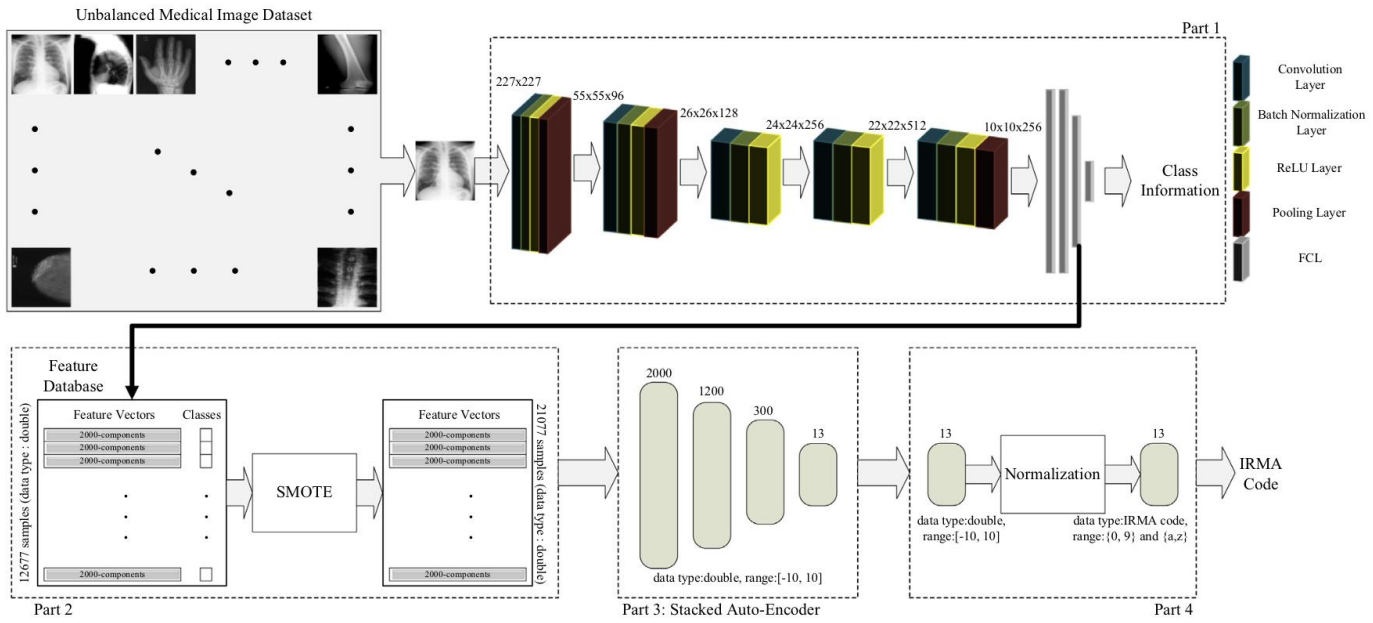
Multimodal Representation Example

- Each modality can be pre-trained
  - using denoising autoencoder
- To train the model, reconstruct both modalities using
  - both Audio & Video
  - just Audio
  - just Video



Ngiam, J., Khosla, A., Kim, M., Nam, J., Lee, H. and Ng, A.Y., 2011. Multimodal deep learning. In *Proceedings of the 28th international conference on machine learning (ICML-11)* (pp. 689-696).

Medical information retrieval



- In the first stage, the convolutional neural network (CNN) architecture is employed to extract discriminative features
- In the second stage, using the Synthetic Minority Over-sampling Technique (SMOTE), the imbalance between the classes in the dataset is reduced.
- Finally, this code is translated to the standard 13-character labeling using stacked autoencoders

Öztürk, Ş., 2020. Stacked auto-encoder based tagging with deep features for content-based medical image retrieval. *Expert Systems with Applications*, *161*, p.113693.

Summary

- Principal components analysis
- Applications using AEs
- Deep AEs
  - Denoising AE
  - Sparse AE
  - Stacked AE
- Modern applications using AEs